SYSTEM ARCHITECTURE

Ownership, Schema, Private Knowledge, Automation & Live Sync

How everything connects — who builds what — how it stays live during the simulation

---

A	B	C	D	E	F
Private Knowledge Flow	Who Builds What	Complete Output Schema	Eval → RAG Pipeline	Staleness Detection	Automated Dashboard

A. Private Knowledge: How It Flows Through the System

Each department has private information that no other department knows. This information serves a specific function at every stage of the simulation. Here is exactly where it appears, how it is used, and why it cannot be circumvented.

A1. The Private Knowledge Each Department Holds

Department	Private Figure	Public Assumption (LLM would use)	Difference & Why It Matters
Finance	Cash reserves: EUR 3,100,000Runway: 11 monthsBurn rate: EUR 282,000/month	EUR 8M raised, ~18 months runway	7-point runway gap completely changes every financial feasibility calculation. The decision to sign Deutsche Bank contract looks safe at 18 months. It looks existential at 11 months.
Engineering	Bias bug: 23% reproduction rate on EU dataAudit trail: does not existInfra gap: EUR 280K and 4 weeks	Product is ready for EU	The bias bug alone blocks compliant launch. Without it, 30-day launch seems feasible. With it, 90 days minimum.
Marketing	True captive waitlist: 300 (not 500)200 clients evaluating HireBot tooTrue ARR potential: EUR 25.5M (not EUR 42.5M)	500 clients waiting, full addressable market	Market opportunity is 40% smaller than stated. Changes whether aggressive compliance spend is justified.
HR	Pipeline: effectively 0.4 FTE at 60 daysCandidate A: competing offer (likely decline)Candidate B: EUR 15K relocation gapCandidate C: junior, 3 months to productivity	3 candidates in pipeline, normal timelines	Hiring plan is nearly impossible at 60-day deadline. Changes every capacity calculation.
Legal	Worst-case fine: EUR 67-80M (not EUR 30M)Praxis AI 2024 precedent (Article 71(4))HireBot challenge: 15-25% success probability	Max fine is EUR 30M (statutory maximum)	Worst-case exposure is 2.5x what everyone assumes. Changes risk-adjusted expected value of non-compliance fundamentally.

A2. Where Private Knowledge Appears in the System

Stage	How Private Knowledge Is Used	What Happens If a Team Skips the Handbook
Building tools	Correct parameter values encode private knowledge. Finance runway calculator must use EUR 3.1M — not found in any public source.	Tool uses EUR 8M. Runway shows 27 months. All downstream decisions based on wrong foundation.

Evaluation scenarios	Eval engine calls tools WITH private parameters as inputs. Scenario F1-H0-A calls calculate_burn_escalation(base_burn=282000, ...). The answer is checked against private-handbook-derived expected output.	Tool returns correct structure but wrong value (used hardcoded public figure). Fails private parameter test.
Department JSON output	known_gaps field should reference gaps created by private knowledge. Legal must flag: 'worst-case fine analysis not included in this output.'	CEO agent acts on incomplete information without knowing it. Information gap detection fails.
CEO synthesis	CEO agent must reason about the probability that departments are withholding information. The private knowledge creates the information asymmetry that makes this reasoning necessary.	CEO treats all outputs as complete. Overestimates confidence in Legal's EUR 12M figure. Makes a misinformed decision.
Board meeting	When department rep meets CEO, they decide how much private knowledge to share. This is a genuine strategic choice — sharing Finance's 11-month runway might cause Marketing to abandon the aggressive launch timeline, which is bad for Marketing KPIs.	If no one reads their handbook, board meeting has nothing strategic to discuss. It becomes a status meeting, not an architectural coordination meeting.
Calibration tracking	After Round 1 evaluation, actual outcomes are revealed. Legal predicted 40% enforcement probability, actual was 60%. The calibration tracker records this. But this only makes sense if Legal's prediction was based on their private analysis, not a generic LLM output.	Calibration tracking records prediction errors, but errors are random (not systematic), so the tracker provides no useful signal.

A3. The Information Boundary Enforcement

The CEO agent cannot read other departments' private RAG folders. This is enforced at the file system level. The only information the CEO receives is what departments include in their structured JSON output. This means:

- If Legal includes the EUR 80M figure in their JSON output, the CEO knows. This was a strategic choice Legal made.
- If Legal does NOT include it, the CEO works with EUR 12M (stated expected value). The eval tests whether the CEO agent appropriately flags this uncertainty.
- Departments can signal gaps without revealing content. Legal can include 'known_gaps: ["worst-case scenario analysis not included"]' without saying the number is EUR 80M. A well-built CEO agent detects this signal and adjusts.

B. Ownership: Who Builds What and How It Connects

This is the most important organizational question of the simulation. The four core components span the entire company but are owned by the CEO team. Here is the complete picture.

B1. Complete Build Ownership Table

Component	Owner Team	Input From	Output To	When Built
Finance Agent + Tools (F1-F4)	Finance team	Private RAG, Global RAG	CEO via dept JSON output	Sprint 1 + evolves every sprint
Engineering Agent + Tools (E1-E4)	Engineering team	Private RAG, Global RAG	CEO via dept JSON output	Sprint 1 + evolves every sprint
Marketing Agent + Tools (M1-M4)	Marketing team	Private RAG, Global RAG	CEO via dept JSON output	Sprint 1 + evolves every sprint
HR Agent + Tools (H1-H4)	HR team	Private RAG, Global RAG	CEO via dept JSON output	Sprint 1 + evolves every sprint
Legal Agent + Tools (L1-L4)	Legal team	Private RAG, Global RAG	CEO via dept JSON output	Sprint 1 + evolves every sprint
Cross-Domain Value Converter	CEO team	Department output schemas (learned in Board Meeting 1)	Consensus Engine	Board Meeting 1 → Sprint 2
Commitment Ledger	CEO team	All department outputs (auto-parsed)	CEO decision, all agents on next run	Sprint 1 stub → Sprint 2 full
Calibration Tracker	CEO team	Eval engine accuracy reports (pushed to Global RAG after each round)	Consensus Engine (adjusts weights)	Sprint 2 → Sprint 3
Consensus Engine	CEO team	Value Converter output, Calibration Tracker weights, Commitment Ledger state	CEO decision JSON	Sprint 1 naive → Sprint 2 designed → Sprint 3 hardened
Global RAG query module	All teams (shared code provided in skeleton)	/mnt/global_rag/ (read-only)	All agents	Provided — students use it
eval_interface.py (submit function)	Provided by instructor — LOCKED	main.py output	/submissions/ folder	Provided — DO NOT MODIFY

B2. How the Four Core Components Connect

The Data Flow Through the CEO System

STEP 1 — Department agents run: Each of the 5 department agents reads the Global RAG, reads its private knowledge folder, calls its calculation tools, and produces a department JSON output. This output is NOT sent directly to the CEO — it is returned to main.py.

STEP 2 — Commitment Ledger check: Before synthesis, the CEO agent queries the Commitment Ledger: 'What has this company committed to in prior rounds that constrains our options right now?' The ledger returns active commitments, their reversibility scores, and any contradictions with the current scenario.

STEP 3 — Value Converter runs: The CEO passes all five department outputs to the Cross-Domain Value Converter. The converter translates every department's output into a single comparable unit (designed by the CEO team in Board Meeting 1). Returns: normalized scores WITH uncertainty ranges for each department.

STEP 4 — Calibration weights applied: The Calibration Tracker adjusts each department's weight based on historical prediction accuracy. A department that has consistently underestimated risk gets down-weighted for risk estimates. A department that has been well-calibrated gets full weight. Returns: calibration-adjusted scores.

STEP 5 — Consensus Engine decides: Takes calibration-adjusted, converted scores. Runs the team's consensus algorithm. Detects preference cycles. Checks for vetoes. Checks commitment ledger contradictions. Returns: final CEO recommendation with full reasoning trace.

STEP 6 — Output assembled and submitted: main.py assembles the complete submission JSON including all department outputs, CEO decision, tools called, and rag_snapshot_hash. Calls submit() from eval_interface.py.

B3. The Board Meeting as Architectural Coordination Mechanism

Board meetings (15 minutes every 1-2 hours, one representative per department + CEO) serve a specific architectural function, not just a status update function. Here is what must happen at each:

Board Meeting	Architectural Purpose	What CEO Team Extracts	What Department Reps Bring
Board Meeting 1 (after Sprint 1, ~12:30 PM)	CEO team learns each department's output schema to design the Cross-Domain Value Converter	For each department: what unit does your main output come in? What is its range? What is your uncertainty model?	Each dept rep brings a printout or screen share of their current JSON output. Points to the specific field the CEO converter needs to read.
Board Meeting 2 (after Round 2 eval, ~2:25 PM)	CEO team reveals calibration accuracy data. Departments learn which of their predictions were wrong and by how much.	Which departments to down-weight in the next round and why. Updated weight parameters for the calibration tracker.	Dept reps explain WHY their Round 1 prediction was wrong — this informs whether the error was random (low information) or systematic (structural bias).

Strategy Break (~4:35 PM)	CEO identifies which component to prioritize for the final sprint. Architecture cards are reviewed.	The single component that would move the score the most. Decision of what to drop vs. what to finish.	Dept reps report: what capability am I missing that would change my output? What am I confident about vs. uncertain about?

C. Complete Output Schema — Given to Students on Day 1

This is the complete schema specification distributed to all students as Handout H-04 on the day of the workshop. Every field is required unless marked OPTIONAL. The eval engine validates this schema as a pre-flight check before running any evaluation logic.

> **Schema Validation is Gate 1**
> Before the evaluation engine runs a single scenario, it validates the submission against this schema. If the schema check fails, the submission is rejected, the deployment penalty counter increments, and the team receives a schema error report. No eval scores are computed. This means teams that do not read the schema carefully will waste deployment attempts on rejected submissions.

C1. Top-Level Submission Schema

TOP-LEVEL SCHEMA

```
# SUBMISSION SCHEMA — eval_interface.py
# Every submission must conform to this schema exactly.
# Pre-flight validation runs before any evaluation logic.

{
  'company_id':           str,    # REQUIRED. 'team_a' | 'team_b' | 'team_c' |
'team_d'
  'submission_number':    int,    # REQUIRED. Increments each submit. Used for
penalty calc.
  'submission_timestamp': str,    # REQUIRED. ISO 8601 format. Set automatically by
submit().
  'active_scenario_id':   str,    # REQUIRED. Read from
/mnt/global_rag/scenario/active_scenario.md
  'rag_snapshot_hash':    str,    # REQUIRED. SHA256 of all files in
/mnt/global_rag/ at time of run.
                                  # Used to detect stale data. Must match current
RAG state ± 10 min.
  'department_outputs': {         # REQUIRED. All 5 departments.
    'finance':     DepartmentOutput,
    'engineering': DepartmentOutput,
    'marketing':   DepartmentOutput,
    'hr':          DepartmentOutput,
    'legal':       DepartmentOutput,
  },
  'ceo_decision': CEODecision,    # REQUIRED.
  'consistency_hash': str,        # REQUIRED. SHA256(department_outputs). Used for
stability test.
                                  # Eval engine runs submission 3 times and checks
this matches.
}
```

C2. DepartmentOutput Schema

DEPARTMENT OUTPUT SCHEMA

```
# DepartmentOutput — one instance per department

DepartmentOutput = {
  # ── IDENTITY ──────────────────────────────────────────────
  'department':           str,     # REQUIRED.
'finance'|'engineering'|'marketing'|'hr'|'legal'
  'agent_model':          str,     # REQUIRED. Ollama model used. e.g.
'llama3.2:3b'

  # ── RECOMMENDATION ────────────────────────────────────────
  'recommendation':       str,     # REQUIRED. One of:
                                   #   'approve'            — launch immediately
                                   #   'conditional_approve'— launch under stated
conditions
                                   #   'delay'              — recommend delay
(state duration)
                                   #   'veto'               — hard block (state
reason)
  'recommended_timeline': str,     # REQUIRED if not 'veto'.
'30d'|'90d'|'Q3'|'conditional'
  'conditions':           [str],   # REQUIRED if 'conditional_approve'. List of
conditions.

  # ── REASONING ─────────────────────────────────────────────
  'stated_reasoning':     str,     # REQUIRED. Min 100 chars. Must reference tool
outputs.
  'key_assumptions':      [str],   # REQUIRED. List of assumptions made. Min 2.

  # ── CONFIDENCE ────────────────────────────────────────────
  'confidence':           float,   # REQUIRED. 0.0 to 1.0.
  'confidence_method':    str,     # REQUIRED. HOW confidence was computed. Not
'LLM estimated'.
                                   # Example: 'Brier-weighted average of tool
output certainties'

  # ── INFORMATION BOUNDARY ──────────────────────────────────
  'information_completeness': float, # REQUIRED. 0.0 to 1.0. Self-reported
completeness.
                                     # 1.0 = shared everything. 0.7 = significant
gaps withheld.
  'known_gaps':          [str],    # REQUIRED. List of known information gaps not
included.
                                   # Can be vague ('additional risk analysis not
included')
                                   # without revealing the actual content.
  'information_withheld': bool,    # REQUIRED. True if information_completeness <
1.0.

  # ── PREDICTIONS (for calibration tracking) ────────────────
  'predictions': {                 # REQUIRED. Department must make quantitative
predictions.
    'prediction_id':   str,        # Unique ID for this prediction.
    'predicted_value': float,      # The predicted value (probability, days, EUR
amount, etc.)
    'prediction_unit': str,        # Unit of the prediction.
    'confidence_interval': [float, float], # [lower_bound, upper_bound]
    'outcome_check_trigger': str,  # When will we know if this was right? e.g.
'after_eval_round_2'
```

```
  },

  # ── TOOLS ──────────────────────────────────────────────────
  'tools_called':       [str],     # REQUIRED. List of tool function names called,
in order.
  'tool_outputs': {                # REQUIRED. Raw output dict from each tool
called.
    '<tool_name>': dict,           # Key = tool name. Value = complete tool return
value.
  },
  'llm_calls_made':     int,       # REQUIRED. Number of Ollama API calls made.

  # ── KPI IMPACT ─────────────────────────────────────────────
  'kpi_impact': {                  # REQUIRED.
    'own_kpi_delta':     float,    # How this recommendation affects department's
own KPIs. -1 to +1.
    'company_kpi_delta': float,    # How this affects overall company KPIs. -1 to
+1.
    'tradeoff_description': str,   # Required if own_kpi_delta and
company_kpi_delta have opposite signs.
  },

  # ── HARD CONSTRAINTS ───────────────────────────────────────
  'hard_constraints':   [str],     # REQUIRED. Non-negotiable conditions dept will
never compromise.
  'veto_conditions':    [str],     # REQUIRED. Exact conditions that trigger a veto
recommendation.
}
```

C3. CEODecision Schema

CEO DECISION SCHEMA

```
# CEODecision — synthesizes all department outputs

CEODecision = {
  # ── DECISION ───────────────────────────────────────────────
  'final_recommendation': str,     # REQUIRED. Same options as
DepartmentOutput.recommendation
  'final_timeline':       str,     # REQUIRED. '30d'|'90d'|'Q3'|'conditional'
  'decision_confidence':  float,   # REQUIRED. 0.0 to 1.0.

  # ── CONSENSUS PROCESS ──────────────────────────────────────
  'consensus_algorithm':  str,     # REQUIRED. Name and brief description of
algorithm used.
  'consensus_ran_n_times': int,    # REQUIRED. How many times consensus was run.
Must be ≥1.
  'consensus_stable':     bool,    # REQUIRED. True if same result across all runs.

  # ── CONFLICT RESOLUTION ────────────────────────────────────
  'conflicts_identified': [str],   # REQUIRED. EVERY disagreement between
departments.
                                   # If 5 departments gave same recommendation,
this is [].
                                   # Typical submission should have 2-4 conflicts
listed.
```

```
  'conflicts_resolved':   [str],   # REQUIRED. HOW each conflict was resolved.
Mechanism, not summary.
  'vetoes_received':      [str],   # REQUIRED. Department names that issued a veto
(if any).
  'veto_overrides':       [str],   # REQUIRED if vetoes overridden. Why each
override is justified.

  # ── VALUE CONVERSION ──────────────────────────────────
  'converter_output': {            # REQUIRED. Cross-Domain Value Converter output.
    '<dept_name>_converted': float, # Converted value for each department's output.
    'conversion_unit':   str,       # Unit of the converted values.
    'uncertainty_ranges': dict,     # Uncertainty range per department.
  },

  # ── CALIBRATION ───────────────────────────────────────
  'calibration_weights_used': {   # REQUIRED from Round 2 onward.
    '<dept_name>': float,          # Weight applied to each department. Sum must =
1.0.
  },

  # ── COMMITMENT LEDGER ─────────────────────────────────
  'commitments_checked':  [str],   # REQUIRED. List of prior commitments checked.
  'contradictions_found': [str],   # REQUIRED. Commitments contradicted by current
scenario.
  'contradiction_resolution': str, # REQUIRED if contradictions_found is non-empty.

  # ── INFORMATION GAPS ──────────────────────────────────
  'information_gaps_flagged': [str], # REQUIRED. Gaps detected in department
outputs.
  'gap_adjusted_confidence':  float, # REQUIRED. Confidence adjusted downward for
known gaps.

  # ── TRACEABILITY ──────────────────────────────────────
  'reasoning':            str,     # REQUIRED. Min 300 chars. Must cite specific
tool outputs
                                   # by tool name AND specific value. Not 'Finance
said X'.
                                   # Must be: 'Finance.calculate_burn_escalation
returned
                                   # runway_months=8.2, therefore...'
  'tools_called':         [str],   # All tools called across ALL agents, in order.
  'consistency_hash':     str,     # SHA256(department_outputs). For stability
testing.
}
```

C4. Schema Validation Pre-Flight Checks

The eval engine runs these checks IN ORDER before any evaluation scenario. First failure = rejection. All failures are listed so teams can fix everything at once.

Check #	What is Validated	Pass Condition	Fail Message

V1	Top-level structure	All 8 top-level keys present and correct type	SCHEMA_ERROR: Missing required field '{field_name}'
V2	All 5 departments present	department_outputs has exactly 5 keys matching exact dept names	SCHEMA_ERROR: Missing department '{dept}' in output
V3	Recommendation values	Each dept recommendation is one of 4 valid values	SCHEMA_ERROR: '{dept}.recommendation' = '{value}' is not valid
V4	Confidence range	Each confidence is float between 0.0 and 1.0 inclusive	SCHEMA_ERROR: '{dept}.confidence' = {value} out of range [0.0, 1.0]
V5	Tools called non-empty	tools_called list has ≥ 1 entry per department	SCHEMA_ERROR: '{dept}.tools_called' is empty — at least one tool must be called
V6	Tool outputs match tools called	Every tool name in tools_called has an entry in tool_outputs	SCHEMA_ERROR: Tool '{tool}' listed in tools_called but missing from tool_outputs
V7	Known gaps list	known_gaps is a list (can be empty, but must be a list)	SCHEMA_ERROR: '{dept}.known_gaps' must be a list, got {type}
V8	Reasoning length	stated_reasoning ≥ 100 chars per dept, CEO reasoning ≥ 300 chars	SCHEMA_ERROR: '{dept}.stated_reasoning' too short ({n} chars, need 100+)
V9	Conflicts listed	CEO conflicts_identified is a list (empty only if all depts agree)	SCHEMA_ERROR: CEO conflicts_identified must be a list
V10	RAG hash present	rag_snapshot_hash is a non-empty string	SCHEMA_ERROR: rag_snapshot_hash missing — agents must query Global RAG and hash it
V11	Consistency hash present	consistency_hash is a non-empty string	SCHEMA_ERROR: consistency_hash missing — required for stability testing
V12	Calibration weights (Round 2+)	If scenario_id is H1 or later, calibration_weights_used must be present	SCHEMA_ERROR: calibration_weights_used required from Hurdle 1 onward

D. Eval → Global RAG Pipeline

After every submission evaluation, the eval engine automatically computes everything that needs to be pushed to the Global RAG. The instructor does not manually calculate any market parameters. The pipeline produces ready-to-push files.

D1. Complete Pipeline Flow

EVALUATION PIPELINE

```
# eval_engine.py — Complete pipeline after a submission

def run_evaluation_pipeline(submission_json_path, team_id):

  # STEP 1 — Load submission and current market state
  submission = load_json(submission_json_path)
  market_state = load_json('/mnt/global_rag/market/market_state.json')
  all_scores = load_json('/mnt/scores/all_scores.json')

  # STEP 2 — Pre-flight schema validation
  schema_errors = validate_schema(submission)
  if schema_errors:
    increment_submission_counter(team_id)  # Counts against penalty
    write_rejection_report(team_id, schema_errors)
    return {'status': 'REJECTED', 'errors': schema_errors}

  # STEP 3 — Staleness detection
  stale_penalty = check_rag_staleness(submission['rag_snapshot_hash'])
  # Returns 0 if fresh, 0.05-0.15 if stale (applied to final score)

  # STEP 4 — Consistency test (run submission 3 times, check hash)
  consistency_score = test_consistency(submission)  # 0.0 to 1.0

  # STEP 5 — Run all evaluation scenarios for active hurdle
  active_hurdle = get_active_hurdle_from_rag()
  scenarios_to_run = get_scenarios_for_hurdle(active_hurdle)
  scenario_scores = {}
  for scenario in scenarios_to_run:
    scenario_scores[scenario.id] = run_scenario(scenario, submission)

  # STEP 6 — Compute dimension scores
  scores = {
    'incommensurability_resolution': score_dimension_1(scenario_scores,
submission),
    'contradiction_detection':       score_dimension_2(scenario_scores,
submission),
    'consensus_stability':           consistency_score * 100,
    'calibration_tracking':          score_dimension_4(scenario_scores,
submission),
    'information_gap_detection':     score_dimension_5(scenario_scores,
submission),
    'decision_traceability':         score_dimension_6(scenario_scores,
submission),
  }
```

```
  weighted_avg = compute_weighted_average(scores)
  penalized_avg = apply_deployment_penalty(weighted_avg, team_id)
  staleness_penalized = penalized_avg * (1 - stale_penalty)

  # STEP 7 — Compute market parameter updates
  market_updates = compute_market_impact(staleness_penalized, market_state,
team_id)

  # STEP 8 — Generate all RAG output files
  bulletin = generate_release_bulletin(team_id, scores, penalized_avg,
market_updates)
  new_market_state = apply_updates(market_state, market_updates)
  new_scoreboard = update_scoreboard(all_scores, team_id, scores)

  # STEP 9 — Write to RAG (instructor confirms before this runs in production)
  push_to_rag('/mnt/global_rag/releases/release_bulletin_NNN.md', bulletin)
  push_to_rag('/mnt/global_rag/market/market_state.json', new_market_state)
  push_to_rag('/mnt/global_rag/scores/scoreboard.md',
render_scoreboard(new_scoreboard))

  return {'status': 'EVALUATED', 'scores': scores, 'market_updates':
market_updates}
```

D2. Market Impact Calculation Formulas

These formulas are hardcoded in eval_engine.py. They compute the market state updates from the submission's weighted average score.

MARKET IMPACT FORMULAS

```
# Market impact formulas — computed automatically after each submission

def compute_market_impact(score_0_to_100, current_market, team_id):

  s = score_0_to_100 / 100  # Normalize to 0-1

  # Market share reduction — higher score = bigger impact (better product)
  share_reduction = 0.04 + (0.08 * s)  # Range: 0.04 (weak) to 0.12 (strong)
  new_market_share = current_market['eu_available_market_share'] * (1 -
share_reduction)
  new_market_share = max(0.10, new_market_share)  # Floor at 10%

  # Market window shrinks with each release
  window_reduction = 0.20 + (0.30 * s)  # 0.20-0.50 months
  new_window = current_market['market_window_months'] - window_reduction
  new_window = max(1.0, new_window)  # Floor at 1 month

  # Regulatory scrutiny increases with market activity
  scrutiny_levels = ['LOW','MEDIUM','MEDIUM_HIGH','HIGH','CRITICAL']
  current_idx = scrutiny_levels.index(current_market['regulatory_scrutiny_level'])
  # Increase scrutiny if score > 60 (strong launch draws attention)
  new_scrutiny_idx = min(4, current_idx + (1 if s > 0.60 else 0))
  new_scrutiny = scrutiny_levels[new_scrutiny_idx]
```

```
  # Deutsche Bank competing offers
  new_db_offers = current_market['deutsche_bank_competing_offers'] + 1

  # HireBot threat increases as market gets competitive
  new_hirebot = min(0.95, current_market['hirebot_threat_score'] + (0.03 + 0.05 *
s))

  # Deployment penalty counter for this team
  submission_n = increment_submission_counter(team_id)
  penalty_rates = {1: 0.0, 2: 0.05, 3: 0.12, 4: 0.22}
  penalty = penalty_rates.get(submission_n, 0.35)

  return {
    'eu_available_market_share': round(new_market_share, 3),
    'market_window_months': round(new_window, 1),
    'regulatory_scrutiny_level': new_scrutiny,
    'deutsche_bank_competing_offers': new_db_offers,
    'hirebot_threat_score': round(new_hirebot, 3),
    'companies_launched_in_eu': current_market['companies_launched_in_eu'] +
[team_id],
    f'{team_id}_submission_count': submission_n,
    f'{team_id}_penalty_rate': penalty,
    'last_release_team': team_id,
    'last_release_score': score_0_to_100,
  }
```

E. Staleness Detection: Ensuring Agents Use Live RAG Data

This is the mechanism that enforces live Global RAG usage. Any agent that hardcodes market parameters into its prompt rather than reading from the Global RAG will fail the staleness test.

E1. How the RAG Snapshot Hash Works

RAG HASH GENERATION — PROVIDED IN STARTER CODE

```
# In agents/department_agent.py — PROVIDED in starter code
# Students must call this at the START of every agent run.

import hashlib, os, json

def read_global_rag_and_hash(rag_path='/mnt/global_rag/'):
  """
  Reads ALL files in the Global RAG folder.
  Returns (content_dict, snapshot_hash).
  content_dict: {filename: content} for all markdown/json files
  snapshot_hash: SHA256 of all content combined — proves when the agent read the
RAG.
  """
  all_content = {}
  all_text = ''
  for root, dirs, files in os.walk(rag_path):
    for f in sorted(files):  # sorted for determinism
      if f.endswith(('.md', '.json')):
        fpath = os.path.join(root, f)
        with open(fpath) as fp:
          content = fp.read()
          all_content[f] = content
          all_text += content
  snapshot_hash = hashlib.sha256(all_text.encode()).hexdigest()
  return all_content, snapshot_hash

# Usage in any department agent:
# rag_content, rag_hash = self.read_global_rag_and_hash()
# market_share = parse_market_param(rag_content, 'eu_available_market_share')
# self.rag_snapshot_hash = rag_hash  # Include in submission output
```

E2. How the Eval Engine Tests for Staleness

STALENESS DETECTION — EVAL ENGINE

```
# In eval_engine.py

def check_rag_staleness(submitted_hash):
  """
  Compares the hash in the submission to the current RAG hash.
  Returns a penalty multiplier: 0.0 = fresh, 0.15 = significantly stale.
```

```
  """
  # Compute current RAG hash
  _, current_hash = read_global_rag_and_hash()

  if submitted_hash == current_hash:
    return 0.0  # Perfect — agent ran with current RAG

  # Check if the submitted hash matches any recent historical RAG state
  # (we keep a rolling log of the last 10 RAG states with timestamps)
  history = load_rag_history()
  for historic_state in history[-10:]:
    if historic_state['hash'] == submitted_hash:
      age_minutes = (now() - historic_state['timestamp']).total_seconds() / 60
      if age_minutes <= 10:
        return 0.0   # RAG was updated very recently — grace period
      elif age_minutes <= 30:
        return 0.05  # Slightly stale — small penalty
      else:
        return 0.15  # Significantly stale — agent did not re-read RAG after update

  # Hash not found in history — agent may have hardcoded values or has a bug
  return 0.20  # Maximum staleness penalty

# IMPORTANT: The eval also checks specific parameter values
def check_parameter_staleness(submission, current_market):
  """
  Checks if specific market parameters used in tool outputs match current values.
  Catches cases where agent reads RAG but ignores updated parameters.
  """
  warnings = []
  for dept, output in submission['department_outputs'].items():
    for tool_name, tool_output in output.get('tool_outputs', {}).items():
      # Check if market share used in marketing tool matches current value
      if 'market_share_input' in tool_output:
        used = tool_output['market_share_input']
        current = current_market['eu_available_market_share']
        if abs(used - current) > 0.05:  # >5% deviation = likely stale
          warnings.append(f'{dept}.{tool_name} used market_share={used},
current={current}')
  return warnings
```

E3. What Students Must Do to Pass Staleness Tests

Instructions for Students (Include in H-05 Setup Instructions)

Your agents MUST read market parameters from the Global RAG at runtime, not from hardcoded values in your code or prompts.

MANDATORY: Call read_global_rag_and_hash() at the start of every agent's analyze() method. Store the returned hash in self.rag_snapshot_hash. Include it in your submission via eval_interface.py.

EXAMPLE — what your Marketing agent should do:

```
rag_content, rag_hash = self.read_global_rag_and_hash()
```

```
market_share = float(parse_rag_param(rag_content, 'eu_available_market_share'))
# Now pass market_share as a parameter to your calculate_market_impact tool
result = self.market_impact_tool.run(eu_available_market_share=market_share, ...)
```

WHAT FAILS: Writing market_share = 1.0 anywhere in your code. Writing 'the EU market has 500 clients' in your prompt. Any value that does not come from the RAG at runtime.

After every hurdle injection, the Global RAG updates. If your agent reads the RAG at runtime, it automatically operates in the new world. If it has hardcoded values, you will get a staleness penalty AND wrong answers.

F. Automated Dashboard & Scoring System

The automated system eliminates manual score tracking and market parameter calculation. The instructor team runs one command after each submission. Everything else is automatic.

F1. System Architecture

INSTRUCTOR SYSTEM STRUCTURE

```
# Directory structure on instructor machine

instructor_system/
├── eval_engine.py          # Core evaluation logic
├── dashboard.py            # Terminal dashboard (run in separate window)
├── submission_watcher.py   # Watches for new submissions, triggers eval
├── rag_manager.py          # Pushes updates to Global RAG
├── penalty_tracker.py      # Tracks submission counts and penalties per team
├── data/
│   ├── market_state.json   # Current market parameters (source of truth)
│   ├── all_scores.json     # All team scores, all rounds
│   ├── submission_log.json # Every submission with timestamp and penalty
│   └── rag_history.json    # Rolling log of RAG states (for staleness check)
└── hurdles/
    ├── hurdle_1.md
    ├── hurdle_2.md
    └── hurdle_3.md
```

F2. Submission Watcher — Runs All Day

SUBMISSION WATCHER

```
# submission_watcher.py — run on instructor machine all day
# python submission_watcher.py

import time, os, subprocess
from pathlib import Path

TEAM_SUBMISSION_DIRS = {
  'team_a': '/mnt/team-a-submissions/',  # NFS mount of team-a machine
  'team_b': '/mnt/team-b-submissions/',
  'team_c': '/mnt/team-c-submissions/',
  'team_d': '/mnt/team-d-submissions/',
}

seen_files = set()

while True:
  for team_id, path in TEAM_SUBMISSION_DIRS.items():
    for f in Path(path).glob('submission_*.json'):
      if str(f) not in seen_files:
        seen_files.add(str(f))
        print(f'\n[{team_id.upper()}] NEW SUBMISSION: {f.name}')
```

```
        print('[INSTRUCTOR] Run eval? (y/n): ', end='')
        if input().strip().lower() == 'y':
          result = subprocess.run(
            ['python', 'eval_engine.py', '--submission', str(f), '--team',
team_id],
            capture_output=True, text=True
          )
          print(result.stdout)
          print('[INSTRUCTOR] Push results to Global RAG? (y/n): ', end='')
          if input().strip().lower() == 'y':
            subprocess.run(['python', 'rag_manager.py', '--push-latest'])
            print('[RAG UPDATED] All teams can now see results.')
  time.sleep(10)  # Check every 10 seconds
```

F3. Terminal Dashboard — Visible on Projector

TERMINAL DASHBOARD OUTPUT

```
# dashboard.py — run in a separate terminal on the projector
# python dashboard.py
# Refreshes every 15 seconds automatically

# OUTPUT EXAMPLE:

# ╔═══════════ OPERATION NEXAAI — LIVE SCOREBOARD ═══════════╗
# ║  14:23  |  Active Hurdle: H1  |  Submissions: 3 total     ║
# ╠═══════════════════════════════════════════════════════════╣
# ║ TEAM       Incomm Contra Stable Calib  InfoGap Trace  AVG  ║
# ║ VeriHire   67     41     58     22     31      77     53.4 ║
# ║ TalentLens  --     --     --     --     --      --     --  ║
# ║ ClearPath   --     --     --     --     --      --     --  ║
# ║ NexaRecruit --     --     --     --     --      --     --  ║
# ╠═══════════════════════════════════════════════════════════╣
# ║ MARKET STATE                                              ║
# ║ EU Market Share Available:  0.88 (was 1.00)               ║
# ║ Market Window:              5.5 months (was 6)            ║
# ║ Regulatory Scrutiny:        MEDIUM-HIGH                   ║
# ║ Deutsche Bank Offers:       1                             ║
# ║ HireBot Threat:             0.52                          ║
# ╠═══════════════════════════════════════════════════════════╣
# ║ SUBMISSION LOG                                            ║
# ║ 14:21 VeriHire  Sub#1  Score:67.4  Penalty:0%  ✓PUSHED    ║
# ║ 14:23 VeriHire  Sub#2  Score:71.2  Penalty:5%  ✓PUSHED    ║
# ╚═══════════════════════════════════════════════════════════╝
# [NEXT HURDLE: H2 at 14:05 — 23 minutes remaining]
```

F4. Hurdle Injection Command

HURDLE INJECTION

```
# To inject a hurdle — instructor runs this one command
# python rag_manager.py --inject-hurdle H1

# rag_manager.py --inject-hurdle logic:
```

```
def inject_hurdle(hurdle_id):
  hurdle_text = read_file(f'hurdles/hurdle_{hurdle_id[-1]}.md')

  # Update active scenario
  write_to_rag('scenario/active_scenario.md', hurdle_text)

  # Append to history
  append_to_rag('scenario/scenario_history.md',
    f'\n\n---\n# INJECTED: {hurdle_id} at {now()}\n\n{hurdle_text}')

  # Update market state with hurdle-specific parameters
  hurdle_market_updates = HURDLE_MARKET_UPDATES[hurdle_id]
  current = load_json('data/market_state.json')
  updated = {**current, **hurdle_market_updates}
  write_to_rag('market/market_state.json', updated)
  save_json('data/market_state.json', updated)

  # Save RAG state to history (for staleness checking)
  save_rag_snapshot_to_history()

  print(f'[HURDLE {hurdle_id} INJECTED] All teams can now read updated RAG.')
  print(f'Market updates applied: {hurdle_market_updates}')

# Hurdle-specific market updates (hardcoded)
HURDLE_MARKET_UPDATES = {
  'H1': {
    'hirebot_eu_status': 'CHALLENGE_FILED',
    'hirebot_threat_score': 0.68,
    'active_eu_enterprise_waitlist': 300,
    'classification_challenge_active': True,
    'regulatory_scrutiny_level': 'MEDIUM_HIGH',
  },
  'H2': {
    'deutsche_bank_status': 'OFFER_ACTIVE',
    'deutsche_bank_contract_value': 10000000,
    'deutsche_bank_deadline_hours': 72,
  },
  'H3': {
    'emergency_directive_active': True,
    'directive_reference': '2026/447',
    'eu_explainability_standard_exists': False,
    'eu_training_data_available': False,
    'deutsche_bank_contract_status': 'UNCERTAIN',
    'regulatory_scrutiny_level': 'CRITICAL',
    'personal_liability_active': True,
  }
}
```

F5. Deployment Penalty Tracker

PENALTY TRACKER

```
# penalty_tracker.py
```

```
PENALTY_SCHEDULE = {
  1: 0.00,   # First release — no penalty
  2: 0.05,   # Second release — 5% reduction
  3: 0.12,   # Third release — 12% reduction
  4: 0.22,   # Fourth release — 22% reduction
  5: 0.35,   # Fifth and beyond — 35% reduction (flat)
}

def get_penalty_and_increment(team_id):
  log = load_json('data/submission_log.json')
  count = len([s for s in log if s['team'] == team_id]) + 1  # +1 for current
  penalty = PENALTY_SCHEDULE.get(count, 0.35)

  log.append({
    'team': team_id,
    'submission_number': count,
    'timestamp': now_iso(),
    'penalty_rate': penalty
  })
  save_json('data/submission_log.json', log)

  print(f'[PENALTY] {team_id} submission #{count}: {penalty*100:.0f}% penalty
applied')
  return count, penalty

def get_team_summary():
  """Returns dict of team_id -> {submissions, current_penalty, total_scores}"""
  # Used by dashboard.py to display deployment history
  ...
```

F6. Accuracy Report: What Gets Pushed After Each Round

After each evaluation round (not just individual releases), the eval engine computes prediction accuracy for each department's Round 1 predictions and pushes an accuracy report to the Global RAG. This feeds the Calibration Tracker.

ACCURACY REPORT FORMAT

```
# accuracy_report pushed to: /mnt/global_rag/scores/prediction_accuracy_round_N.md

# EXAMPLE — after Round 1 evaluation
# This is what the calibration tracker reads to update department weights

# PREDICTION ACCURACY REPORT — Round 1
# Generated by eval_engine.py after Round 1 evaluation

# FINANCE
#   Predicted: EU revenue over 18 months = EUR 8.3M
#   Actual (simulated): EUR 6.1M
#   Error: +36% overestimate
#   Calibration score: 0.64 (lower = less accurate)
#   Error direction: systematic overestimation of revenue

# LEGAL
```

```
#   Predicted: enforcement probability if launched in 30 days = 0.40
#   Actual (simulated): 0.60
#   Error: -33% underestimate of risk
#   Calibration score: 0.67
#   Error direction: systematic underestimation of regulatory risk

# ENGINEERING
#   Predicted: system reliability under EU load = 0.73
#   Actual (simulated): 0.81
#   Error: -10% underestimate (small)
#   Calibration score: 0.90
#   Error direction: slight underestimation

# MARKETING
#   Predicted: market share loss if delayed to Q3 = 15pp
#   Actual (simulated): 8pp
#   Error: +88% overestimate of competitive threat
#   Calibration score: 0.52
#   Error direction: systematic overestimation of competitor threat

# HR
#   Predicted: time to hire 4 compliance engineers = 12 weeks
#   Actual (simulated): 18 weeks
#   Error: -33% underestimate of hiring difficulty
#   Calibration score: 0.67
#   Error direction: systematic underestimation of hiring friction

# RECOMMENDED CALIBRATION WEIGHTS FOR ROUND 2:
# Finance:     0.17  (was 0.20, down for revenue overestimation)
# Engineering: 0.24  (was 0.20, up for good calibration)
# Marketing:   0.14  (was 0.20, down for large overestimation)
# HR:          0.18  (was 0.20, down slightly)
# Legal:       0.27  (was 0.20, up — but apply asymmetric correction for risk
underestimation)
# NOTE: Legal underestimating risk is the most serious error type. Weight increased
# but with asymmetric correction: Legal's risk estimates should be multiplied by
1.5
# when used by the CEO consensus engine.
```

How Teams Should Use the Accuracy Report

The Calibration Tracker should read /mnt/global_rag/scores/prediction_accuracy_round_N.md automatically after each round. The weights in the recommended section are not binding — teams should design their own weight update formula. The recommended weights are a baseline that a naive implementation would use. A team that designs an asymmetric loss function (penalizing risk underestimation more than overestimation for Legal) will outperform the naive baseline.

This is exactly the kind of domain expertise insertion point the simulation is designed around: an HR professional knows that HR systematically underestimates hiring timelines, so the correction factor for HR estimates should be applied asymmetrically upward. A finance professional knows the same about revenue projections. These corrections cannot be looked up — they come from professional experience.

Operation NexaAI — System Architecture, Ownership & Automation

INSTRUCTOR ONLY — Advanced Agentic AI Program